

FungalMorphoSpace v0.6.2

Documentación técnica (desde mi código): ecuaciones, parámetros y efectos

January 2, 2026

1 Propósito

Este documento describe mi implementación de morfogénesis tipo Turing para himenóforos poroides. La idea operativa es simple: yo fijo una cinética de reacción (por defecto Schnakenberg), fijo un régimen de difusión relativa (D_v/D_u) y calibro dos “perillas” principales:

- ρ como control de escala (“zoom”) del espaciado característico del patrón.
- b como control geométrico/topológico (riesgo de cambiar puntos por laberintos o rayas).

Mi pipeline está diseñado para ser auditable: todo lo que ejecuto queda registrado en tablas (humana y máquina), y el simulador decide dt_{final} de forma estable mediante un criterio CFL para difusión.

2 Arquitectura del proyecto (lo que hace cada pieza)

Mi flujo “integrated validation” tiene tres capas:

1. **Base de especies y presets:** `data/species_data.json` y el loader `src/fungalmorphospace/runners/species_loader.py`. Aquí defino presets por especie (parámetros y metadatos) y resuelvo alias a keys canónicas.
2. **Simulación:** `src/fungalmorphospace/core/turing_simulator.py`. Aquí integro el sistema RD en una grilla 2D, con control de paso temporal y guardado del patrón.
3. **Análisis y reporte:** `src/fungalmorphospace/analysis/topology_analyzer.py` y `src/fungalmorphospace/reporting.py`. Aquí mido longitudes de onda (λ) por FFT y autocorrelación, detecto “spots”, calculo métricas topológicas, y genero `COMPREHENSIVE_COMPARISON.png` y tablas `validation_summary*.csv/json`.

2.1 CLI: el punto de entrada

Yo ejecuto el pipeline con:

```
python3 scripts/run_integrated_validation.py --species all --n_runs 1
```

El argumento `--species` acepta keys cortas canónicas (por ejemplo `fomes`, `brumalis`, `squamosus`) y también alias largos (por ejemplo `polyporus_squamosus`). Internamente, normalizo todo a la key canónica.

3 Mi base de especies: keys canónicas y alias

3.1 Diseño del JSON

En `data/species_data.json` yo declaro:

- **Key canónica corta** (ej.: `squamosus`).
- **Nombre científico** (ej.: *Polyporus squamosus*).
- **Role** (ACTIVE o EXCLUDED). Mantengo `trametes` como EXCLUDED para que exista como referencia, pero no corra en `--species all`.
- **Aliases**: lista de strings aceptadas por CLI.
- **Parameters**: bloque anidado con presets (esto es crítico).

3.2 Bug clásico evitado: parámetros anidados

Yo guardo los presets dentro de `"parameters": {...}`. Si el loader intenta leer `D_v_D_u` o `rho` desde el top-level, cae a defaults y todas las especies terminan con el mismo conjunto de parámetros. Mi loader correcto siempre hace:

```
params = species_data.get("parameters", {})  
DvDu = float(params.get("D_v_D_u", 150.0))  
rho = float(params.get("rho", 0.2))  
...
```

Esa regla es la que protege la calibración por especie.

4 Modelo matemático que implemento

4.1 Ecuaciones RD con factor de velocidad ρ

Mi simulador integra dos campos en 2D (activador u e inhibidor v):

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u + \rho f(u, v), \quad (1)$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + \rho g(u, v). \quad (2)$$

En mi código trabajo frecuentemente con el cociente D_v/D_u porque captura el principio LALIP: inhibición a largo alcance.

4.2 Cinética Schnakenberg (mi baseline)

En el modo Schnakenberg (baseline de mis presets), uso:

$$f(u, v) = a - u + u^2 v, \quad (3)$$

$$g(u, v) = b - u^2 v. \quad (4)$$

El parámetro b desplaza el equilibrio (u_0, v_0) y modifica el Jacobiano local. Esto es la fuente de su efecto geométrico: cambia qué modos crecen y qué atractor no lineal domina (spots vs laberintos vs rayas).

5 Cómo interpreto las perillas (lo que realmente controlan en mi pipeline)

5.1 ρ : mi control de escala (“zoom”)

La razón por la cual trato ρ como “zoom” es que se puede absorber en un reescalamiento temporal. Defino $\tau = \rho t$, entonces:

$$\frac{\partial}{\partial t} = \rho \frac{\partial}{\partial \tau}.$$

Sustituyendo:

$$\frac{\partial u}{\partial \tau} = \frac{D_u}{\rho} \nabla^2 u + f(u, v), \quad (5)$$

$$\frac{\partial v}{\partial \tau} = \frac{D_v}{\rho} \nabla^2 v + g(u, v). \quad (6)$$

En otras palabras, mi λ característica escala aproximadamente como:

$$\lambda \propto \sqrt{\frac{D}{\rho}} \Rightarrow \boxed{\lambda \propto \frac{1}{\sqrt{\rho}} \text{ si } D \text{ fijo}}.$$

Consecuencia práctica en mi código: si una especie ya está en spots (poros redondos) pero la densidad (pores/mm) no cuadra, ajustar ρ cambia el tamaño/espaciado sin necesidad de alterar la topología.

5.2 b : mi control geométrico (alto riesgo)

En mi pipeline, b cambia el equilibrio y el Jacobiano J de la cinética; eso desplaza el sistema en el “espacio de Turing”. Por eso lo trato como parámetro de riesgo:

- Pequeños cambios pueden transformar spots en laberintos.
- También puedo salir del rango de inestabilidad y perder el patrón.

Si mi objetivo es mantener “identidad poroide” (spots), yo fijo b en el régimen ya validado y calibro escala con ρ .

5.3 D_v/D_u : mi palanca de LALIP

El cociente D_v/D_u determina cuán “de largo alcance” es la inhibición. En mi calibración:

- Si el sistema no entra a spots, ajusto D_v/D_u (y en último caso b) para reentrar a un régimen de Turing apropiado.
- Si ya hay spots, prefiero tocar ρ antes que b .

6 Control numérico: $dt_{\text{requested}}$, CFL y dt_{final}

6.1 Por qué yo reajusto dt

Mi simulador aplica un criterio de estabilidad para difusión (CFL). Para una discretización explícita típica en 2D:

$$dt \leq \frac{\Delta x^2}{4D_{\max}}.$$

Yo implemento esto en forma de:

$$dt_{\max}^{\text{diff}} = \frac{\Delta x^2}{4D_{\max}}, \quad dt_{\max}^{\text{safe}} = s dt_{\max}^{\text{diff}},$$

con un *safety factor* $s \in (0, 1)$. Luego:

$$dt_{\text{final}} = \min(dt_{\text{requested}}, dt_{\max}^{\text{safe}}).$$

En mis tablas exporto `dt_requested`, `dt_final`, `dt_max_diffusion` y `safety_factor` para que el ajuste sea auditable.

6.2 Cap de pasos para ejecución reproducible: `--max_steps`

Cuando D_v/D_u es enorme (caso *squamosus*), dt_{final} puede ser muy pequeño y el número de pasos se dispara. Para que mi smoke test sea estable, introduzco un cap:

$$N_{\text{steps, nominal}} = \left\lceil \frac{T_{\text{target}}}{dt_{\text{final}}} \right\rceil, \quad N_{\text{steps}} = \min(N_{\text{steps, nominal}}, N_{\text{max}}).$$

Luego:

$$T_{\text{actual}} = N_{\text{steps}} dt_{\text{final}}.$$

Esto permite correr `--species all` con `--grid 256` y tener duraciones comparables entre especies, sin perder trazabilidad.

7 Medición de longitud de onda y por qué tengo “dos λ ”

En mi análisis exporto tres valores:

- λ_{FFT} : pico dominante del espectro de potencia.
- λ_{AC} : estimación por autocorrelación espacial.
- λ_{best} : selección robusta (elige FFT si pasa QC; si no, autocorr).

Esto no es redundancia: cuando el patrón es ruidoso, anisótropo o tiene banda ancha, FFT y autocorr pueden divergir. Por eso guardo ambos y reporto `wavelength_method` y `wavelength_qc_pass`.

8 Salidas: por qué exporto dos CSV

Mi reporte genera:

- `results/tables/validation_summary.csv` (humano): nombres amigables, redondeos, rangos como texto.
- `results/tables/validation_summary_machine.csv` (máquina): schema estable en `snake_case`, tipos numéricos, columnas de auditoría (`dt`, `steps`, `QC`, `paths`).

Yo uso el CSV “máquina” como contrato para:

- smoke tests (detección de regresiones),
- scripts de plots (por ejemplo escoger `wavelength_best_px` vs `wavelength_fft_px`),
- pipelines de comparación y validación automatizada.

El CSV “humano” es para lectura y comunicación (figuras, documentos, revisión rápida).

9 Smoke test endurecido: qué valida exactamente

Mi smoke test no solo ejecuta, sino que verifica invariantes:

1. **Contrato de columnas:** el machine CSV debe contener `dt`, `CFL` y `QC`.
2. **Diversidad paramétrica:** D_v/D_u , b y ρ deben coincidir con presets por especie (evita el bug de defaults).
3. **Consistencia temporal:** $T_{\text{actual}} = N_{\text{steps}} dt_{\text{final}}$.
4. **Artefactos:** cada fila debe apuntar a un `pattern_path` existente y, si aplica, a una copia canónica.

Ese conjunto de checks convierte el smoke test en un guardrail real, no solo en “el script no se cayó”.

10 Reglas de calibración que sigo (operativas)

Objetivo	Perilla principal	Riesgo / nota
Mantener spots y cambiar densidad	Ajustar ρ	Bajo. Control de escala.
Cambiar régimen (entrar/salir de spots)	Ajustar D_v/D_u	Medio. Afecta ventana Turing.
Cambiar geometría/topología	Ajustar b	Alto. Puede producir laberintos o perder patrón.
Estabilidad numérica	dt_{final} por <code>CFL</code> ; <code>cap</code> por <code>--max.steps</code>	Es un tema de integración, no de biología.

Table 1: Reglas prácticas de cómo calibro mi simulador.

11 Nota final: qué significa “me dio todo lo mismo”

Si varias especies corren con los mismos D_v/D_u , b y ρ , y además se reusa el mismo seed, el patrón y sus métricas tienden a coincidir. En mi proyecto, eso no se interpreta como “las especies son iguales”, sino como una alarma de integridad de presets (loader cayendo en defaults). Por eso el smoke test endurecido valida diversidad paramétrica contra el JSON.